1. Introduction

Dans ce projet, nous avons implémenté quatre algorithmes de recherche de chemin :

- BFS (Breadth First Search)
- Dijkstra
- Recherche gloutonne (Greedy Best-First Search)
- A* (A-star)

Le but est de comparer leurs performances sur différentes cartes en analysant :

- La distance trouvée
- Le nombre d'états explorés
- Le temps d'exécution

2. Description des algorithmes

2.1 BFS

BFS explore les états niveau par niveau en utilisant une file (FIFO).

- Il garantit un chemin optimal si tous les coûts sont identiques.
- Il explore souvent beaucoup d'états.
- Il ne prend pas en compte les coûts différents des cases.

2.2 Dijkstra

Dijkstra est un algorithme de plus court chemin pour des graphes pondérés.

- Il garantit toujours un chemin optimal.
- Il prend en compte les coûts de déplacement.
- Il explore beaucoup d'états.
- Il est plus lent que BFS.

2.2 Dijkstra

Dijkstra est un algorithme de plus court chemin pour des graphes pondérés.

- Il garantit toujours un chemin optimal.
- Il prend en compte les coûts de déplacement.

- Il explore beaucoup d’états.
- Il est plus lent que BFS.

2.3 Recherche gloutonne (Greedy)

La recherche gloutonne utilise uniquement une heuristique.

Elle choisit toujours le nœud qui semble le plus proche de l’objectif.

- Très rapide
- Explore peu d’états
- Ne garantit pas un chemin optimal

3. Cartes utilisées

Nous avons utilisé trois cartes différentes :

1. 8room_000.map
2. Paris_2_256.map
3. 32room_000.map

Ces cartes ont été choisies pour tester les algorithmes sur :

- Une carte simple
- Une carte urbaine réaliste
- Une carte plus complexe avec plusieurs salles

4. Résultats expérimentaux

Carte 8room

```
julia> include("src/main.jl")
Carte étudiée : dat/8room_000.map
BFS ALGO
Distance D → A : 48
Number of states evaluated : 1748
Path D → A : [(100, 100), (101, 100), (102, 100), (103, 100), (103, 99), (103, 98), (103, 97), (103, 96), (104, 96), (105, 96), (106, 96), (107, 96), (108, 96), (109, 96), (110, 9
6), (111, 96), (112, 96), (112, 97), (112, 98), (112, 99), (113, 99), (114, 99), (115, 99), (116, 99), (117, 99), (117, 100), (117, 101), (117, 102), (117, 103), (117, 104), (117,
 105), (117, 106), (118, 106), (119, 106), (119, 107), (119, 108), (119, 109), (119, 110), (119, 111), (119, 112), (119, 113), (119, 114), (120, 114), (120, 115), (120, 116), (120
, 117), (120, 118), (120, 119), (120, 120)]
Carte étudiée : dat/8room_000.map
Dijkstra algo
Distance D → A : 48.0
Number of states evaluated : 1798
Path D → A : [(100, 100), (101, 100), (102, 100), (102, 99), (102, 98), (103, 98), (103, 97), (103, 96), (104, 96), (105, 96), (106, 96), (107, 96), (108, 96), (109, 96), (110, 96
), (111, 96), (112, 96), (112, 97), (112, 98), (112, 99), (113, 99), (114, 99), (115, 99), (115, 100), (115, 101), (116, 101), (116, 102), (117, 102), (117, 103), (117, 104), (117
, 105), (117, 106), (117, 107), (117, 108), (117, 109), (117, 110), (117, 111), (118, 111), (119, 111), (119, 112), (119, 113), (119, 114), (119, 115), (119, 116), (119, 117), (11
9, 118), (119, 119), (120, 119), (120, 120)]
Carte étudiée : dat/8room_000.map
Gluton algo
Distance D → A : 56
Number of states evaluated : 102
Path D → A : [(100, 100), (101, 100), (102, 100), (103, 100), (103, 99), (103, 98), (103, 97), (103, 96), (104, 96), (105, 96), (106, 96), (107, 96), (108, 96), (109, 96), (110, 9
6), (111, 96), (112, 96), (112, 97), (112, 98), (112, 99), (113, 99), (114, 99), (115, 99), (116, 99), (117, 99), (118, 99), (119, 99), (120, 99), (120, 100), (120, 101), (120, 10
2), (120, 103), (120, 104), (119, 104), (118, 104), (117, 104), (117, 105), (117, 106), (118, 106), (119, 106), (120, 106), (120, 107), (120, 108), (120, 109), (120, 110), (120, 1
11), (120, 112), (119, 112), (119, 113), (119, 114), (120, 114), (120, 115), (120, 116), (120, 117), (120, 118), (120, 119), (120, 120)]
Carte étudiée : dat/8room_000.map
astra ALGO
Distance D → A : 48.0
Number of states evaluated : 158
Path D → A : [(100, 100), (101, 100), (102, 100), (103, 100), (103, 99), (103, 98), (103, 97), (103, 96), (104, 96), (105, 96), (106, 96), (107, 96), (108, 96), (109, 96), (110, 9
6), (111, 96), (112, 96), (112, 97), (112, 98), (112, 99), (113, 99), (114, 99), (115, 99), (116, 99), (117, 99), (117, 100), (117, 101), (117, 102), (117, 103), (117, 104), (117,
 105), (117, 106), (118, 106), (119, 106), (119, 107), (119, 108), (119, 109), (119, 110), (119, 111), (119, 112), (119, 113), (119, 114), (120, 114), (120, 115), (120, 116), (120
, 117), (120, 118), (120, 119), (120, 120)]
```

Observation :

- BFS et Dijkstra trouvent la distance optimale.
- Glouton trouve un chemin plus long.
- A* explore beaucoup moins d’états.

Carte Paris

```
Carte étudiée : dat/Paris_2_256.map
BFS ALGO
Distance D → A : 46
Number of states evaluated : 1301
Path D → A : [(100, 100), (101, 100), (101, 99), (102, 99), (103, 99), (103, 98), (104, 98), (105, 98), (106, 98), (106, 97), (107, 97), (108, 97), (109, 97), (110, 97), (111, 97)
, (112, 97), (113, 97), (114, 97), (114, 98), (115, 98), (116, 98), (117, 98), (117, 99), (117, 100), (117, 101), (117, 102), (117, 103), (117, 104), (117, 105), (117, 106), (118,
 106), (118, 107), (119, 107), (119, 108), (120, 108), (120, 109), (120, 110), (120, 111), (120, 112), (120, 113), (120, 114), (120, 115), (120, 116), (120, 117), (120, 118), (120
, 119), (120, 120)]
Carte étudiée : dat/Paris_2_256.map
Dijkstra algo
Distance D → A : 46.0
Number of states evaluated : 1310
Path D → A : [(100, 100), (100, 99), (101, 99), (102, 99), (102, 98), (103, 98), (104, 98), (105, 98), (106, 98), (106, 97), (107, 97), (108, 97), (108, 98), (109, 98), (109, 99),
 (109, 100), (109, 101), (109, 102), (110, 102), (110, 103), (110, 104), (111, 104), (112, 104), (113, 104), (113, 105), (113, 106), (113, 107), (114, 107), (115, 107), (116, 107)
, (116, 108), (116, 109), (116, 110), (116, 111), (116, 112), (116, 113), (116, 114), (117, 114), (118, 114), (118, 115), (119, 115), (119, 116), (119, 117), (119, 118), (119, 119
), (120, 119), (120, 120)]
Carte étudiée : dat/Paris_2_256.map
Gluton algo
Distance D → A : 52
Number of states evaluated : 55
Path D → A : [(100, 100), (101, 100), (101, 99), (102, 99), (103, 99), (103, 98), (104, 98), (105, 98), (106, 98), (106, 97), (107, 97), (108, 97), (109, 97), (110, 97), (111, 97)
, (112, 97), (113, 97), (114, 97), (114, 98), (115, 98), (116, 98), (117, 98), (118, 98), (119, 98), (120, 98), (120, 99), (120, 100), (120, 101), (119, 101), (119, 102), (118, 10
2), (118, 103), (118, 104), (117, 104), (117, 105), (117, 106), (118, 106), (118, 107), (119, 107), (119, 108), (120, 108), (120, 109), (120, 110), (120, 111), (120, 112), (120, 1
13), (120, 114), (120, 115), (120, 116), (120, 117), (120, 118), (120, 119), (120, 120)]
Carte étudiée : dat/Paris_2_256.map
astra ALGO
Distance D → A : 46.0
Number of states evaluated : 218
Path D → A : [(100, 100), (101, 100), (101, 99), (102, 99), (103, 99), (103, 98), (104, 98), (105, 98), (106, 98), (106, 97), (107, 97), (108, 97), (109, 97), (110, 97), (111, 97)
, (112, 97), (113, 97), (114, 97), (114, 98), (115, 98), (116, 98), (117, 98), (117, 99), (117, 100), (117, 101), (117, 102), (117, 103), (117, 104), (117, 105), (117, 106), (118,
 106), (118, 107), (119, 107), (119, 108), (120, 108), (120, 109), (120, 110), (120, 111), (120, 112), (120, 113), (120, 114), (120, 115), (120, 116), (120, 117), (120, 118), (120
, 119), (120, 120)]
```

Observation :

- Même comportement.
- A* est beaucoup plus efficace que BFS et Dijkstra.

Carte 32room

```
Carte étudiée : dat/32room_000.map
BFS ALGO
Distance D → A : 40
Number of states evaluated : 1357
Path D → A : [(100, 100), (101, 100), (102, 100), (103, 100), (104, 100), (105, 100), (106, 100), (107, 100), (108, 100), (109, 100), (110, 100), (111, 100), (112, 100), (113, 100
), (114, 100), (115, 100), (116, 100), (117, 100), (118, 100), (119, 100), (120, 100), (120, 101), (120, 102), (120, 103), (120, 104), (120, 105), (120, 106), (120, 107), (120, 10
8), (120, 109), (120, 110), (120, 111), (120, 112), (120, 113), (120, 114), (120, 115), (120, 116), (120, 117), (120, 118), (120, 119), (120, 120)]
Carte étudiée : dat/32room_000.map
Dijkstra algo
Distance D → A : 40.0
Number of states evaluated : 1387
Path D → A : [(100, 100), (100, 101), (100, 102), (100, 103), (100, 104), (100, 105), (101, 105), (102, 105), (103, 105), (103, 106), (104, 106), (104, 107), (104, 108), (105, 108
), (105, 109), (106, 109), (107, 109), (108, 109), (109, 109), (110, 109), (110, 110), (110, 111), (110, 112), (110, 113), (110, 114), (111, 114), (112, 114), (113, 114), (113, 11
5), (113, 116), (114, 116), (114, 117), (115, 117), (115, 118), (116, 118), (116, 119), (117, 119), (118, 119), (119, 119), (120, 119), (120, 120)]
Carte étudiée : dat/32room_000.map
Gluton algo
Distance D → A : 40
Number of states evaluated : 41
Path D → A : [(100, 100), (101, 100), (102, 100), (103, 100), (104, 100), (105, 100), (106, 100), (107, 100), (108, 100), (109, 100), (110, 100), (111, 100), (112, 100), (113, 100
), (114, 100), (115, 100), (116, 100), (117, 100), (118, 100), (119, 100), (120, 100), (120, 101), (120, 102), (120, 103), (120, 104), (120, 105), (120, 106), (120, 107), (120, 10
8), (120, 109), (120, 110), (120, 111), (120, 112), (120, 113), (120, 114), (120, 115), (120, 116), (120, 117), (120, 118), (120, 119), (120, 120)]
Carte étudiée : dat/32room_000.map
astra ALGO
Distance D → A : 40.0
Number of states evaluated : 441
Path D → A : [(100, 100), (101, 100), (102, 100), (103, 100), (104, 100), (105, 100), (106, 100), (107, 100), (108, 100), (109, 100), (110, 100), (111, 100), (112, 100), (113, 100
), (114, 100), (115, 100), (116, 100), (117, 100), (118, 100), (119, 100), (120, 100), (120, 101), (120, 102), (120, 103), (120, 104), (120, 105), (120, 106), (120, 107), (120, 10
8), (120, 109), (120, 110), (120, 111), (120, 112), (120, 113), (120, 114), (120, 115), (120, 116), (120, 117), (120, 118), (120, 119), (120, 120)]
(40.0, 441, [(100, 100), (101, 100), (102, 100), (103, 100), (104, 100), (105, 100), (106, 100), (107, 100), (108, 100), (109, 100) … (120, 111), (120, 112), (120, 113), (120, 1
14), (120, 115), (120, 116), (120, 117), (120, 118), (120, 119), (120, 120)])

julia>
```

Observation :

- Glouton trouve ici le chemin optimal.
- A* reste plus efficace que BFS et Dijkstra.

5. Analyse comparative

On observe que :

- BFS explore beaucoup d'états.
- Dijkstra explore encore plus d'états à cause des coûts.
- Glouton est très rapide mais peut produire un chemin non optimal.
- A* est le meilleur compromis : il est rapide et trouve toujours le chemin optimal.

A* est donc l'algorithme le plus efficace dans notre projet.

6. Conclusion

Dans ce projet, nous avons comparé quatre algorithmes de recherche de chemin.

Les résultats montrent que :

- BFS et Dijkstra garantissent l'optimalité mais sont coûteux.
- La recherche gloutonne est rapide mais non fiable.
- A* est l'algorithme le plus performant en pratique.

Ainsi, A* représente le meilleur choix pour résoudre des problèmes de recherche de chemin sur des cartes complexes